

On typing knowledge graphs

Iztok Savnik¹

Department of computer science,
Faculty of mathematics, natural sciences and information technologies,
University of Primorska, Slovenia
`iztok.savnik@upr.si`

Abstract. This paper presents a formal type system for knowledge graphs (KGs). The schema of a KG represents a language of types comprising the type identifiers—such as classes and predicates—ordered into taxonomies, together with the product, intersection, and union types. To handle the rich hierarchy of types, the typing framework employs algebraic operators on posets to formalize typing relations. Typing of ground triples proceeds by first inferring and minimizing the ground type from the stored typing of entities, then generalizing it to a minimal upper bound (MUB) type. The MUB type serves as a basis for relating ground types to the schema of a KG. Next, the schema type is inferred from the predicate’s definition, minimized, and filtered through subtyping with respect to the MUB ground type to produce the final schema type. The proposed typing rules support type inference, provide a framework for type-checking of KGs and graph patterns, and enable predicate disambiguation based on the triple types.

Keywords: knowledge representation · knowledge graphs · type systems .

1 Introduction

A knowledge graph (abbr., KG) is a formalism for representing data and knowledge in the form of a graph. The vocabularies, such as RDF-Schema [27], attach the meanings to the edges of a graph, and turn a graph into a language for the representation of data and knowledge. The expressive power of a KG can be observed from KG’s relation to logic. From this perspective, a KG is a set of named binary relations, and triples are logical statements. Let us present some of the features of the data and knowledge representation language of a KG.

A KG stores predefined classes that are organized into taxonomies [2]. Further, typing of ground identifiers, as well as typing of the predicates with the domain and range types, is also stored in a KG. Similarly to the *roles* [4] of a knowledge base, the predicates are treated as classes. As classes, predicates are organized into a taxonomy. However, they also act as individual entities described with additional predicates. Finally, the predicates are inherited through the taxonomy of classes similarly as data members and methods are inherited among classes of object-oriented programming languages. And since predicates also act as classes, the predicates of the predicates, like the domain and range of a predicate, are inherited through the taxonomy of predicates.

From a logic perspective, the schema of a KG represents the definitions of concepts (classes) and binary relations (predicates); this is often referred to as the intensional

knowledge. On the other hand, the ground entities (identifiers) and ground triples represent the facts about the entities, or the extensional knowledge.

From the perspective of a type system, the schema of a KG is presented by using a language of types [24], often the RDF-Schema. The syntax of type expressions includes the following structures. First, we have classes that are the types of ground identifiers (entities). Second, the predicates are the identifiers (names) of binary relations. The types of predicates are the product types, including the domain and range types and the associated predicate name. Further, the $\wedge$ - $\vee$ types [23] capture many requirements of the KG domain very naturally. For example, a domain of a predicate can be two different classes that are linked by an $\wedge$ -type.

Finally, the taxonomies of classes and predicates are hard to grasp structurally; rather, posets can be manipulated with algebraic operators. In the proposed type system, we use the operators defined on posets in rules. Examples include the operator *minimum*, which, given a poset, returns a set of minimal poset elements, and the operator for computing *minimal upper bounds* of a subset of elements from a poset [6].

The paper presents a type system comprising a set of rules for typing ground triples from a KG. Let us give an abstract description of typing a single ground triple t . First, the *ground type* of t is inferred bottom-up, from the stored typing of ground entities, which are the components of t . The ground type is further minimized to obtain a minimal ground type of t . The minimal ground type is then generalized to obtain a minimal upper bound (MUB) type. The MUB type is an appropriate starting point for exploring its relations to the conceptual schema of a knowledge graph. Second, the *schema type* of t is inferred from the conceptual schema based on the predicate of t . All valid schema types of t are gathered and then minimized to obtain a schema type with a minimal interpretation. Finally, the minimal schema type is filtered by relating it to the MUB ground type via the subtype relation to compute the *final schema type* of t .

The contributions of the presented research are as follows. First, to our knowledge, this is the first proposal for typing ground triples of a knowledge graph that uses the product, union and intersection types, and encompasses a complete KG when typing a ground triple. Second, the presented framework for typing ground triples can serve conveniently for the implementation of type-checking the stored typing of a KG. The analysis of typing a ground triple spans from the ground types at the lower levels of ontology to the minimal schema types at the upper levels of ontology. Finally, we show that the triple types can be used to disambiguate the sense of a predicate with multiple meanings.

The paper is organized as follows. The following Section 2 presents definitions of an RDF graph, a KG data model, and the typing rule language. Next, typing of identifiers is presented in Section 3. Here we present typing literals and introduce typing and subtyping of identifiers. In Section 4 we present the semantics of $\wedge$ - $\vee$ types, the associated rules for typing and subtyping, and introduce the join and meet types. Typing of triples is described in Section 5. We present the derivation of ground and schema triple types, and the rules to combine these two into the final type. Section 6 presents an overview of the related work on typing KGs. Finally, in Section 7 we discuss some directions of further work.

2 Preliminaries

In this Section, we present the definition of an RDF graph, a data model of a KG, and the typing rule language that we use in the sequel.

2.1 RDF graph

An RDF graph [26] is defined as follows. Let I be a set of URI-s, B be a set of blanks, and L be a set of literals. Let us also define sets $S = I \cup B$, $P = I$, and $O = I \cup B \cup L$. An *RDF triple* is a triple $(s, p, o) \in S \times P \times O$. An *RDF graph* $g \subseteq S \times P \times O$ is a set of triples. A set of all graphs will be denoted as G .

The complete set of triples of an RDF graph is in the text referred to as Δ . We use Δ when we want to refer to the original set of KG triples and not the data model of a KG presented in the following Section.

2.2 A data model of a KG

This Section defines a knowledge graph as an RDF graph that uses the RDF-Schema vocabulary [27] for the representation of the schema part of a knowledge graph. To abstract away the details of the RDF data model, we unify the representation of the knowledge graph by separating solely between the *identifiers* and the *triples*.

The identifiers include the set of ground (individual) identifiers, i.e., identifiers that denote individual entities, and the set of type identifiers, i.e., the identifiers that denote the types of individual entities. The triples include the set of ground triples and the set of triple types.

However, typing of a KG is based on the separation between the values $\mathcal{V}$ and the types $\mathcal{T}$. The set of values $\mathcal{V}$ includes the ground identifiers $\mathcal{V}_i$ and the ground triples $\mathcal{V}_t$. To be able to refer to the specific subsets of $\mathcal{V}_i$ in the rules, we also introduce the set of literal values $\mathcal{V}_l \subseteq \mathcal{V}_i$, and the set of predicates $\mathcal{V}_p \subseteq \mathcal{V}_i$. The set of all valid types $\mathcal{T}$ comprises the type identifiers $\mathcal{T}_i$, i.e., types of ground identifiers $\mathcal{V}_i$, and the triple types $\mathcal{T}_t$, i.e., the types of ground triples $\mathcal{V}_t$. Finally, a set $\mathcal{T}_c \subseteq \mathcal{T}_i$ is a set of type (class) identifiers, and a set $\mathcal{T}_p \subseteq \mathcal{T}_i$ denotes a set of predicates that now have the role of types.

The set of type identifiers $\mathcal{T}_i$ is ordered by a subtype relation $\preceq$ to form a poset. The meaning of type identifiers is established by their interpretations. The interpretation of a type identifier $T_i \in \mathcal{T}_i$ is the set of its instances, including the instances of type identifiers $T'_i \preceq T_i$.

The ground triples include solely the ground identifiers from $\mathcal{V}_i$ in places of S and O, and predicates $\mathcal{V}_p$ in the place of P component of a triple. The types of ground triples are the product types $T_s * p * T_o$ where $T_s \in \mathcal{T}_c$, $T_o \in \mathcal{T}_c \cup \mathcal{T}_l$, and $p \in \mathcal{T}_p$. The product types are in our data model, written as triples (T_s, p, T_o) . In Section 4, we extend triple types with $\wedge$ - $\vee$ types in the subject and object components.

The triple types $\mathcal{T}_t$ are ordered by a subtype relation $\preceq$ to form a poset. The subtype relation $\preceq$ among the triple types is induced from a subtype relation defined among the type identifiers $\mathcal{T}_i$. A more detailed formal definition of a KG is given in [31].

2.3 Typing rule language

We depart from the standard typing-rule notation with contexts (Γ) [24, 11] and instead employ a meta-language rooted in first-order logic (FOL). This meta-language is inspired by Pierce's presentation of typing rules [24], especially in the treatment of records, subtyping, and universal quantification.

The rules are composed of a set of premises and a conclusion. The premises can be expressions stating the set membership of objects, typing and subtyping judgements, expressions of the FOL, or set definition statements. FOL expressions can express complex premises, such as the conditions for the LUB types. The conclusion part of the rule is a typing or subtyping judgment that can include a set definition statement.

The symbols used in a rule are grounded by stating their membership in the sets of identifiers ($\mathcal{V}$, $\mathcal{V}_i$, and $\mathcal{V}_t$) and triples ($\mathcal{T}$, $\mathcal{T}_i$ and $\mathcal{T}_t$) defined in Section 2.1. When we write $o \in s$, then we mean the *existence* of an object o in a set s . We use universal quantification $\forall o \in s, p(o)$ when we state that some property $p(o)$ holds for all objects o from s . The premises of the rule are treated from left to right. The quantification of the symbols binds the symbols up to the last premise unless defined differently by the parentheses.

The usual typing relation $x : T$, saying that x is of type T , can be annotated to obtain a specific typing relation $x :_* T$, where $*$ stands for any annotation. All annotated typing relations $x :_* T$ imply a usual typing relation $x : T$. The annotation is needed since we construct the final type of a triple t from two different types, the ground type $t :_\downarrow T_g$ and schema type $t :_\uparrow T_s$. Moreover, the annotation is also employed to guide the application of some specific rules. All the annotated typing relations are introduced before their use.

3 Typing identifiers

In this section, we present the typing of ground identifiers $\mathcal{V}_i$. First, typing of literals $\mathcal{V}_l$ is described in Section 3.1. Next, the rules for ground typing and subtyping of ground identifiers are presented in Section 3.2. Section 3.3 further extends ground typing and subtyping to the complete set of type identifiers $\mathcal{T}_i$.

3.1 Typing literals

Literal values $\mathcal{V}_l \subseteq \mathcal{V}_i$ are the instances of literal types $\mathcal{T}_l \subseteq \mathcal{T}_i$. The literal types $\mathcal{T}_l$ are provided by the RDF-Schema vocabulary [27]. RDF-Schema defines a rich set of literal types, such as `xsd:integer`, `xsd:string`, or `xsd:boolean`.

The literals are composed of literal values and literal types. For example, the literal `"365"^^xsd:integer` includes the literal value `"365"` and a type `xsd:integer`. Typing of literals is defined by the following rule.

$$\frac{L \in \mathcal{V}_l \quad T \in \mathcal{T}_l \quad \text{"L"}^{\wedge\wedge T} \in \Delta}{L : T} \quad (1)$$

The rule states that a literal value L is of a type T if a literal `"L"^^T` is an element of a KG Δ . A literal type T is referencing a type from the RDF-Schema vocabulary.

3.2 Stored typing and subtyping of identifiers

The typing expression $V :_1 T$ denotes a *stored typing* relation $:_1$ that links a value $V \in \mathcal{V}_i$ to a ground type identifiers $T \in \mathcal{T}_i$. The stored typing is a one-step typing relation based on the triples stored in a KG.

The stored types are linked to the ground identifiers via stored triples $(I, \text{rdf:type}, T) \in \Delta$. Consequently, we can write $I :_1 T$. The rule for the one-step stored typing relation $:_1$ is defined as follows.

$$\frac{I \in \mathcal{V}_i \quad T \in \mathcal{T}_i \quad (I, \text{rdf:type}, T) \in \Delta}{I :_1 T} \quad (\text{I-GND})$$

The individual entity I can have more than one stored type. Using Rule ?? as a generator gives all types $T_j^{j \in [1, n]}$ such that $I :_1 T_j^{j \in [1, n]}$.

If we want to obtain all valid stored types of I , Rule ?? can be used either in some other rule that employs it as a generator, or we can update the above rule to generate an $\wedge$ -type, including all the stored types of I as presented in Section 4.

The one-step subtyping relation $\preceq_1$ is defined on classes by using the RDF predicate `rdfs:subClassOf` as follows.

$$\frac{C_1, C_2 \in \mathcal{T}_c \quad (C_1, \text{rdfs:subClassOf}, C_2) \in \Delta}{C_1 \preceq_1 C_2} \quad (2)$$

The rule for the definition of the one-step subtyping relationship $\preceq_1$ on the predicates $\mathcal{T}_p$ is based on the RDF-Schema predicate `rdfs:subPropertyOf`.

$$\frac{P_1, P_2 \in \mathcal{T}_p \quad (P_1, \text{rdfs:subPropertyOf}, P_2) \in \Delta}{P_1 \preceq_1 P_2} \quad (3)$$

The predicates have in the above rule the role of types in the sense that they represent the names of the binary relations.

3.3 Typing and subtyping identifiers

The one-step relation $\preceq_1$ is extended with the reflectivity, transitivity, and antisymmetry to obtain the subtyping relation $\preceq$. The relation $\preceq$ forms a poset of type identifiers $\mathcal{T}_i$.

First, the one-step relation $\preceq_1$ is generalized to the relation $\preceq$ defined over type identifiers $\mathcal{T}_i$.

$$\frac{I_1, I_2 \in \mathcal{T}_i \quad I_1 \preceq_1 I_2}{I_1 \preceq I_2} \quad (4)$$

The following Rules 5-7 define the reflexivity, transitivity and antisymmetry of the relation $\preceq$, respectively.

$$\frac{I \in \mathcal{T}_i}{I \preceq I} \quad (5)$$

$$\frac{I_1, I_2, I_3 \in \mathcal{T}_i \quad I_1 \preceq I_2 \quad I_2 \preceq I_3}{I_1 \preceq I_3} \quad (6)$$

$$\frac{I_1, I_2 \in \mathcal{T}_i \quad I_1 \preceq I_2 \quad I_2 \preceq I_1}{I_1 = I_2} \quad (7)$$

As a consequence of Rules 5-7, the relation $\preceq$ orders the types from $\mathcal{T}_i$ into a poset.

The one-step typing relation $:_1$ is generalized to the typing relation $:$ with the following rule.

$$\frac{I \in \mathcal{V}_i \quad T \in \mathcal{T}_i \quad I :_1 T}{I : T} \quad (8)$$

Note that a unique notation for stored typing allows us to address differently the *stored* and the *derived* types of ground identifiers.

The link between the typing relation $:$ and the subtype relation $\preceq$ is provided by adding a typing rule called the *rule of subsumption* [24].

$$\frac{I \in \mathcal{T}_i \quad S, T \in \mathcal{T}_i \quad I : S \quad S \preceq T}{I : T} \quad (\text{I-SUB})$$

Finally, KGs include a special class $\top$ that represents the root type of an ontology. In RDF ontologies, $\top$ is usually represented by the predicate `owl:Thing` [12]. The following rule specifies that all types are subtypes of $\top$.

$$\frac{\forall S \in \mathcal{T}}{S \preceq \top} \quad (9)$$

4 Intersection and union types

The rules for the $\wedge$ and $\vee$ -types presented in this section are general—they apply for the type identifiers $\mathcal{T}_i$ and triple types $\mathcal{T}_t$.

The meaning of the $\wedge$ and $\vee$ -types can be seen through their interpretations. The instances of the intersection type $T_1 \wedge T_2$ are objects belonging to both T_1 and T_2 . The type $T_1 \wedge T_2$ is the greatest lower bound of the types T_1 and T_2 . In general, $\bigwedge_{i=1}^n T_i$ is the greatest lower bound (abbr. GLB) of types T_i $i=1..n$ [22, 23]. The instances of the type $\bigwedge_{i=1}^n T_i$ form a maximal set of objects that belong to all types T_i .

The union type is dual to the intersection type. The instances of the union type $T_1 \vee T_2$ are objects from the interpretations of both types, T_1 and T_2 . The type $T_1 \vee T_2$ denotes the least upper bound of types T_1 and T_2 . A general form of union type is $\bigvee_{i=1}^n T_i$. The interpretation of $\bigvee_{i=1}^n T_i$ represent the minimal set of instances that include the instances of all types T_i $i=1..n$.

The interpretation of a type $\bigwedge_{i=1}^n T_i$ is included in interpretation of every particular type T_i which means $\bigwedge_{i=1}^n T_i \preceq T_i$. This is stated by the following rule.

$$\frac{T_i \text{ } i=1..n \in \mathcal{T}}{\bigwedge_{i=1}^n T_i \preceq T_i \text{ } i=1..n} \quad (10)$$

Further, the following rule states that if the type S is a subtype of every type T_i $i=1..n$ then S is a subtype of $\bigwedge_{i=1}^n T_i$.

$$\frac{S \in \mathcal{T} \quad T_i^{i=1..n} \in \mathcal{T} \quad S \preceq T_i^{i=1..n}}{S \preceq \bigwedge_{i=1}^n T_i} \quad (11)$$

The opposite of the above rule, the following rule states the necessary conditions that must be met so that a type T is a supertype of a type $\bigwedge_{i=1}^n S_i$. It is enough that there is one type $S_i \preceq T$ for the intersection of $S_i^{i=1..n}$ to be included in T .

$$\frac{T \in \mathcal{T} \quad S_i^{i=1..n} \in \mathcal{T} \quad S \in \{S_i\}_{i=1}^n \quad S \preceq T}{\bigwedge_{i=1}^n S_i \preceq T} \quad (12)$$

The duality of the intersection and union types can also be seen from the duality of the rules for the $\wedge$ and $\vee$ -types.

The union type $\bigvee_{i=1}^n T_i$ is the least upper bound of types $T_i^{i=1..n}$ [22]. This means that all types $T_i^{i=1..n}$ are subtypes of their union.

$$\frac{T_i^{i=1..n} \in \mathcal{T}}{T_i^{i=1..n} \preceq \bigvee_{i=1}^n T_i} \quad (13)$$

The following rule defines the necessary conditions to be met for a type T to be a supertype of a type $\bigvee_{i=1}^n S_i$.

$$\frac{T \in \mathcal{T} \quad S_i^{i=1..n} \in \mathcal{T} \quad S_i^{i=1..n} \preceq T}{\bigvee_{i=1}^n S_i \preceq T} \quad (14)$$

Again, the opposite rule defines the premises that must hold so that S is a subtype of a type $\bigvee_{i=1}^n T_i$.

$$\frac{S \in \mathcal{T} \quad T_i^{i=1..n} \in \mathcal{T} \quad T \in \{T_i\}_{i=1}^n \quad S \preceq T}{S \preceq \bigvee_{i=1}^n T_i} \quad (15)$$

Note that besides checking the subtype relation between a type treated as a whole and some logical type, Rules 10-15 can be used to check the subtyping among arbitrary logical types.

4.1 The join and meet types

The $\vee$ and $\wedge$ types are logical types defined on the basis of their interpretations, i.e., the sets of instances. Given two types T and S we have a least upper bound $S \vee T$, and a greatest lower bound $S \wedge T$ types where $S \vee T$ denotes a minimal set of objects that are of type S or T (or both), and $S \wedge T$ denotes a maximal set of objects that are of type S and T .

A KG includes a stored poset of classes and triple types that represent types of the individual identifiers and ground triples. The poset can be used to compute a join $S \sqcup T$ and a meet $S \sqcap T$ of the parameter types. The usual definition of the join and meet operators is by using a least upper bound and a greatest lower bound if they exist [24], respectively. However, in a KG, we are also interested in the upper bound and lower bound types [6]. Let us present an example.

Example 1. Let $P = (U, \preceq)$ be a partially ordered set such that $U = \{a, b, c, d, e\}$ and the relation $\preceq = \{a \preceq c, a \preceq d, b \preceq c, b \preceq d, c \preceq e, d \preceq e\}$. The upper bounds of $S = \{a, b\}$ are the elements c and d . Since there are no lower upper bounds, the upper bounds $\{c, d\}$ are minimal upper bounds. The least upper bound of S is e .

In the case that we remove the element e from P , then P does not have a least upper bound, but it still has two minimal upper bounds c and d . $\square$

The least upper bound (LUB) and the greatest lower bound (GLB) of a set are one element by definition. On the other hand, the most interesting upper and lower bounds are the minimal upper bound (MUB) and maximal lower bound (MLB) [17]. They can be lower than the least upper bound and higher than the greatest lower bound, respectively. They represent more detailed information about the parameter set of types S than the LUB type of S .

The join $J = \sqcup_{i=1}^n T_i$ is a set of MUB types $J_j^{j \in [1, m]} \in J$ such that J_j is an upper bound with $T_i^{i=1..n} \preceq J_j$, and there is no such L where $T_i^{i=1..n} \preceq L$ without also having $J_j \preceq L$. Since we have a top type $\top$ defined in a KG, the join of two arbitrary types always exist.

The meet of types $T_i^{i=1..n}$, $M = \cap_{i=1}^n T_i$, is a set of the MLB types $M_j^{j \in [1, m]} \in M$ such that M_j is a lower bound with $M_j \preceq T_i^{i=1..n}$, and all other lower bounds U with $U \preceq T_i^{i=1..n}$ entail $U \preceq M_j$. Note that the meet of the set of types from a KG does not always exist.

The join type is related to the $\vee$ -type. Given a set of types $\{T_i\}_{i=1}^n$, the join $J = \sqcup_{i=1}^n T_i$ is a set of types $J_j^{j \in [1, m]} \in J$ that are the minimal upper bounds such that $T_i^{i=1..n} \preceq J_j$. On the other hand, Rule 13 for the $\vee$ -types states $T_i^{i=1..n} \preceq \vee_{i=1}^n T_i$. However, the join type and $\vee$ -type differ in the interpretation.

$$\llbracket \vee_{i=1}^n T_i \rrbracket_{\Delta} = \bigcup_{i=1}^n \llbracket T_i \rrbracket_{\Delta} \subseteq \bigcap_{j=1}^m \llbracket J_j \rrbracket_{\Delta} = \llbracket \sqcup_{i=1}^n T_i \rrbracket_{\Delta}$$

While the interpretation of the type $\vee_{i=1}^n T_i$ includes precisely the instances of all T_i , the interpretation of the types $J_j \in J = \sqcup_{i=1}^n T_i$ contains interpretations of $T_i^{i=1..n}$ since $T_i^{i=1..n} \preceq J_j^{j=1..m}$. Hence, $\cup_{i=1}^n \llbracket T_i \rrbracket_{\Delta} \subseteq \cap_{j=1}^m \llbracket J_j \rrbracket_{\Delta}$. The interpretation of $J_j^{j=1..m}$ can include besides the interpretations of $T_i^{i=1..n}$ also the interpretations of some other classes.

A meet type of $T_i^{i=1..n}$ may not exist in a poset of types from a KG. In general, the meet types $M = \cap_{i=1}^n T_i$ exist in a class ontology if the types $T_i^{i=1..n}$ are *bounded below* [24] which means that there exists a type L such that $L \preceq T_i^{i=1..n}$. The meet types are not frequent on the lower levels of a class ontology from a KG.

As in the case of the $\vee$ -type and the join type, the semantics of the $\wedge$ -type is comparable to the semantics of the meet type. An $\wedge$ -type is a type that implements logical view of the greatest lower bound type. The meet types are based on the concrete poset of KG types and represent concrete types. Their interpretation is contained in the interpretation of a $\wedge$ -type. The type $\wedge_{i=1}^n T_i$ denotes the intersection $\cap_{i=1}^n \llbracket T_i \rrbracket_{\Delta}$ while the interpretation of a meet types $M_{j=1}^m \in \cap_{i=1}^n T_i$ includes the union $\cup_{j=1}^m \llbracket M_j \rrbracket_{\Delta}$. Since $M_{j=1}^m \preceq T_{i=1}^n$, then $\llbracket M_{j=1}^m \rrbracket_{\Delta} \subseteq \llbracket T_{i=1}^n \rrbracket_{\Delta}$. Hence, $\cup_{j=1}^m \llbracket M_j \rrbracket_{\Delta} \subseteq \llbracket \cap_{i=1}^n T_i \rrbracket_{\Delta}$.

Note that the instances of the meet types $M_{j=1}^m$ are in the intersection of the instances of the types $T_{i=1}^n$. The set $\cap_{i=1}^n \llbracket T_i \rrbracket_\Delta$ can also include objects that are not instances of any meet type from $M_{j=1}^m$. Hence,

$$\llbracket \bigwedge_{i=1}^n T_i \rrbracket_\Delta = \bigcap_{i=1}^n \llbracket T_i \rrbracket_\Delta \supseteq \bigcup_{j=1}^m \llbracket M_j \rrbracket_\Delta = \llbracket \cap_{i=1}^n T_i \rrbracket_\Delta.$$

When typing the ground triples, the join types are used in the procedure for checking the types derived bottom-up against the stored schema of a KG as presented in Section 5.2. The join as well as meet types are useful in the procedure for type-checking basic graph patterns [30]. The $\vee$ and $\wedge$ -types are logical types that can be simplified in the typing positions of a graph pattern by using typing rules, and can be approximated by using join and meet types to obtain a *concrete* type of a graph pattern variable.

4.2 Typing identifiers with $\wedge$ - $\vee$ types

In this section, we first gather the classes that are the ground types of an individual (ground) identifier $I \in \mathcal{V}_i$ by using an $\wedge$ -type in Rule L-GND. Since a user defines the stored ground types of a ground identifier, the stored types may include subclasses of other stored types. Therefore, the minimal type T_{min} is first determined by eliminating the redundant class identifiers from T in Rule L-MIN. Finally, from the minimal ground type T_{min} of I we derive the minimal upper bound (MUB) by Rule ???. The MUB type is the starting point for investigating the relations between the ground type and the schema type of a triple t as presented in Section 5.

We start with a rule for gathering the ground types of a ground identifier $I \in \mathcal{V}_i$. The ground types are gathered in the premise of the rule by selecting individual ground types as presented in Section 3.2.

$$\frac{I \in \mathcal{V}_i \quad T_i^{i=1..n} \in \mathcal{T}_i \quad I : T_i^{i=1..n}}{I : \bigwedge_{i=1}^n T_i} \quad (\text{L-GND})$$

The ground type $\bigwedge_{i=1}^n T_i$ can include pairs of types $T_i \preceq T_k$ with $i \neq k$. Depending on the further use, we can either infer the *minimal* or the *maximal elements* of $\{T_i\}_{i=1}^n$ with respect to $\preceq$ [6]. The supertypes of the minimal elements of $\{T_i\}_{i=1}^n$ include all valid types of I . Hence, we use the set of minimal elements from $\{T_i\}_{i=1}^n$ as the starting point to explore the relations between the ground types and the schema types of a triple t .

Given a set of types $P = \{T_i\}_{i=1}^n$ ordered by $\preceq$, the minimal elements of P are the elements $S_j^{j=1..m} \in \{T_i\}_{i=1}^n$ such that $\nexists T_i^{i=1..n} (T_i \prec S_j)$. All pairs of types $S_k, S_l \in \{S_j\}_{j=1}^m$ with $k \neq l$ are *incomparable*, i.e., $S_k \not\prec S_l \equiv S_k \not\preceq S_l \wedge S_k \not\preceq S_l$. The logical rule for deriving a minimal type of a ground type $\bigwedge_{i=1}^n T_i$ is defined as follows.

$$\frac{I \in \mathcal{V}_i \quad I : \bigwedge_{i=1}^n T_i \quad S_1, S_2 \in \{T_i\}_{i=1}^n, S_1 \preceq S_2}{I : \downarrow \wedge \{S \mid S \in \{T_i\}_{i=1}^n \wedge \forall i \in [1, n], S \preceq T_i \vee S \not\prec T_i\}} \quad (\text{L-MIN})$$

Note that we used the annotated typing relation " $\downarrow$ " to denote that we would like to infer the minimal type of I .

Let us now present the typing rule that, given $I \in \mathcal{V}_i$ and $I : \downarrow \wedge_{i=1}^n T_i$, derives the join ground type of I . Recall from Section 4.1 that we defined the operation join as the MUB of a set $\{T_i\}_{i=1}^n$. A join $\sqcup_{i=1}^n T_i$ is a set of minimal upper bounds $\{S_j\}_{j=1}^m$ that are related to all types T_i $^{/i \in [1, n]}$ via $\preceq$, and are minimal. Below is the logical definition of the join rule.

$$\frac{I \in \mathcal{V}_i, I : \wedge_{i=1}^n T_i}{I :_{\sqcup} \wedge \{S \mid S \in \mathcal{T}_i, T_i^{i=1..n} \preceq S \wedge \forall P \in \mathcal{T}_i, ((T_i^{i=1..n} \preceq P \wedge S \preceq P) \vee P \not\preceq S)\}} \quad (16)$$

The annotated typing relation $:_{\sqcup}$ is used in the conclusion of the above rule to be able to explicitly require the use of the rule in other rules. For example, Rule 16 is used for the definition of Rule 20, which is used for the derivation of the join ground type of a ground triple.

The join ground types are used as the starting points for searching a correct schema type. In case a predicate has two alternative definitions in two different contexts, then a path from an MUB type to the schema of a KG determines the corresponding schema type. The details are presented in Section 5.4.

5 Typing triples

This section presents the typing of triples. First, in Section 5.1 we describe the product type and its role as a triple type. In the following sub-sections, the rules for typing a ground triple $t = (s, p, o)$ are presented. We use two different types to describe t . The *ground type* of t is based on the stored typing of the ground identifiers. The rules for deriving the ground type are presented in Section 5.2. The *schema type* of t is derived from the stored conceptual schema of t . The rules for the derivation of the schema type are given in Section 5.3. To be able to distinguish the rules for the derivation of a ground type from the rules for deriving the schema type of t , we denote the ground typing relation as $:_{\downarrow}$, and the schema typing relation as $:_{\uparrow}$. After computation of the ground and schema types of t the final type of t is determined by relating the schema and ground types. This is presented in Section 5.4.

5.1 Triple types

A type of a triple $t = (s, p, o) \in \mathcal{V}_t$ is a product type $D * p * R$ where $s : D$ and $o : R$ holds. In our model, the triple types are represented by a triple $(D, p, R) \in \mathcal{T}_t$. The types D and R are type expressions that represent either a class identifier or an expression composed of classes related by $\wedge$ and/or $\vee$ operators. The available logical operators depend on the schema language of a KG. For now, we assume the schema language is RDF-Schema.

The $\wedge$ -types reflect the semantics of RDF-Schema [27] which permits the definition of multiple domains and ranges of the predicate p that are linked with $\wedge$ -types. Consequently, each predicate p has exactly one triple type of the form

$$(\wedge_{i=1}^n D_i, p, \wedge_{j=1}^m R_j).$$

Example 2. If p has two domains D_1 and D_2 , and a single range type R then the type corresponding to p is $(D_1 \wedge D_2, p, R)$. $\square$

The interpretation of a triple type (D, p, R) is defined as follows.

$$\llbracket (D, p, R) \rrbracket_\Delta = \{(s, p, o) \mid s \in \llbracket D \rrbracket_\Delta \wedge o \in \llbracket R \rrbracket_\Delta\}$$

The subtype relationship among the triple types is defined on the basis of the subtype relationship among classes, $\wedge$ and $\vee$ -types, and predicates. The following rule defines the relationship $\preceq$ between two triple types.

$$\frac{T_1 \in \mathcal{T}_t, T_1 = (D_1, p_1, R_1) \quad T_2 \in \mathcal{T}_t, T_2 = (D_2, p_2, R_2) \quad D_1 \preceq D_2 \quad p_1 \preceq p_2 \quad R_1 \preceq R_2}{T_1 \preceq T_2} \quad (17)$$

The above Rule 17 handles the $\wedge$ -types of the subject and object through Rules 10-12.

5.2 Ground types of a triple

The ground types of a triple t are either a stored ground type, a minimal ground type, or a join ground type. The stored ground type includes types that are stored in a KG. The minimal ground type then consists solely of the minimal types from the stored ground types. Finally, the join ground type is the conjunction of minimal upper bound types obtained by applying the MUB operator to the conjunction of minimal ground types [17].

A ground type of a ground identifier I is a class identifier C related to I by one-step type relationship $:_1$, as presented by Rule I-GND. In terms of the concepts of a knowledge graph, I and C are related by the relation `rdf:type`.

A ground type of a triple $t = (I_s, p, I_o)$ is a product type $S * p * T$ that we represent as a triple (S, p, T) . A triple type includes the ground types of t 's components I_s and I_o , and the property p , which now has the role of a type. The following rule defines a ground type of a triple.

$$\frac{t \in \mathcal{V}_t, t = (I_s, p, I_o) \quad S, T \in \mathcal{T}_i \quad I_s :_1 S \quad I_o :_1 T \quad p : \text{rdf:Property}}{t :_\downarrow (S, p, T)} \quad (3\text{-GND})$$

Note that Rule I-SUB is a generator that enumerates all possible one-step ground types of t . All other derived ground types of t can be derived using Rule I-SUB.

The types S and T are either class identifiers or $\wedge$ -types composed of a conjunction of class identifiers. The predicates are treated differently from the subject and object components of triples. The predicates have the role of types while they are instances of `rdf:Property`.

The minimal ground type of a triple t can be obtained by computing the minimal ground types of the triple components.

$$\frac{t \in \mathcal{V}_t, t = (I_s, p, I_o) \quad S, T \in \mathcal{T}_i \quad I_s :_\downarrow S \quad I_o :_\downarrow T \quad p : \text{rdf:Property}}{t :_\downarrow (S, p, T)} \quad (18)$$

The component types S and T of the minimal ground type (S, p, T) can represent $\wedge$ -types. The following rule transforms a triple type including $\wedge$ -types into an $\wedge$ -type of simple triple types composed of class identifiers in place of the subject and object components. The rule is expressed in a general form by using the typing relation ":", which can be used instead of any annotated typing relation.

$$\frac{t \in \mathcal{V}_t \quad t : (\wedge_{i=1}^n S_i, p, \wedge_{j=1}^m T_j)}{t : \wedge_{i=1, j=1}^{n, m} (S_i, p, T_j)} \quad (19)$$

The type in the conclusion of the rule is constructed by the Cartesian product of the sets of types belonging to the types of the subject and object components. Since each of the types S_i and T_j is valid for the subject and object components of t , respectively, then also the triple types from the conclusion of the rule are valid.

The above decomposition of a triple type into a set of triple types is useful when we check the ground types against the schema triple types to select the valid schema triple type of a triple. This is detailed in Section 5.3.

Finally, the join of a set of ground types is a set of MUB types, as defined in Section 4.1. Similar to the previous rules for ground typing, the join is derived by inferring the join types of the subject and object components of t .

$$\frac{t \in \mathcal{V}_t, t = (I_s, p, I_o) \quad S, T \in \mathcal{T}_i \quad I_s :_{\sqcup} S \quad I_o :_{\sqcup} T \quad p : \text{rdf:Property}}{t :_{\downarrow} (S, p, T)} \quad (20)$$

The join type (S, p, T) includes in the subject and object components the $\wedge$ -types composed of one or multiple MUB classes, i.e., $(\wedge_{i=1}^n S_i, p, \wedge_{j=1}^m T_j)$. We convert this type using Rule 19 into a conjunction of basic MUB triple types $M = \wedge_{i=1..n, j=1..m} (S_i, p, T_j)$ to be used as a *final ground type*.

We can easily see that each particular triple type from M is an MUB type since it includes MUB types in its components. Further, the MUB types from M are incomparable since the MUB types of the components are incomparable by $\preceq$.

All rules defined for the ground triple types rely on inferring the types of their components. The reason for this is the stored typing of ground identifiers as well as the stored subtype relation $\preceq$. Both are defined solely for classes and predicates.

5.3 Schema triple types

The schema types are types of triples defined by a variant of KG schema. The schema definition language currently used in KGs is either RDF-Schema [27] or RDF-Schema combined with OWL [10] vocabulary. In this section, we present the semantics of both approaches.

We do not expect that the domain and range of the predicate are defined for each particular predicate p . They can be inherited from the super-predicates of p . Hence, a predicate p has the domain and range defined either directly, when domain and range are defined for the predicate p , or indirectly, if the domain and range are defined for p 's super-predicates and inherited by the predicate p .

The rules for the derivation of the schema triple type of a given triple t are presented for two different schema definition languages. In Section 5.3.1 we present the derivation of schema types when the RDF-Schema is used. Further, in Section 5.3.2 we define the typing rules for KGs that use RDF-Schema together with $\wedge$ - $\vee$ types.

KGs with RDF-Schema. When the semantics of a KG is defined by using RDF-Schema, we can specify one or more domain and range types, but no alternative types. The semantics of RDF-Schema [28] interprets multiple domains and ranges with $\wedge$ -type. If p has two domains T_1 and T_2 then p can link subjects I that are of type T_1 and T_2 . The domain type of p is then $T_1 \wedge T_2$, or, in terms of OWL [21], `owl:intersectionOf( $T_1$   $T_2$ )`. The RDF-Schema does not allow the definition of the domain or range of a predicate with the $\vee$ -type. Consequently, each predicate can have only one meaning.

We first determine *all* valid schema types for a given triple $t = (s, p, o)$. A schema triple type comprises a predicate p and the domain and range types linked to predicates $p' \succeq p$. The domain and range types can be defined for the predicate p and/or inherited from the predicates $p' \succ p$. In addition, the domain and range types can be, in general, inherited from two different super-predicates of p .

$$\frac{\begin{array}{l} t \in \mathcal{V}_i, t = (s, p, o) \quad p_1, p_2 \in \mathcal{T}_p, p \preceq p_1 \preceq p_2 \\ T \in \mathcal{T}_c, (p_1, \text{domain}, T) \in \Delta \\ S \in \mathcal{T}_c, (p_2, \text{range}, S) \in \Delta \end{array}}{t :_{\uparrow} (T, p, S)} \quad (21)$$

The above rule generates valid schema types (T, p, S) for a given predicate p . We allow that a domain and range types are defined for different predicates $p_1, p_2 \succeq p$ since such cases appear in a KG. However, we have to be careful that the rules of inheritance are respected. We can only inherit from p_1 and p_2 , which are related by a subtype relation: $p \preceq p_1 \preceq p_2$. This condition restricts the domain and range to be defined on the same path from p to some maximal element m of the predicate p poset.

Rule 21 generates all valid schema types of a triple t . From the set of all valid stored types of t , we select the subset including only the minimal types. The following rule is a logical judgment for a minimal schema type of t .

$$\frac{\begin{array}{l} t \in \mathcal{T}_i \quad T_i^{i=1..n} \in \mathcal{T}_t, t :_{\uparrow} T_i \\ S_j^{j=1..m} \in \mathcal{T}_t, t :_{\uparrow} S_j \quad T_i \preceq S_j \vee T_i \not\preceq S_j \end{array}}{t :_{\uparrow} \wedge_{i=1}^n T_i} \quad (22)$$

The first part of the premise says that t is a ground triple and there exists $T_i^{i=1..n} \in \mathcal{T}_t$, which are the types of t . The second part of the premise requires that $T_i^{i=1..n}$ are the minimal types. In other words, there are no such types $S_j^{j=1..m}$ of t that are subtypes of $T_i^{i=1..n}$. Hence, $T_i^{i=1..n}$ are the minimal types of the stored triple types of t .

Note that if the schema is defined by using RDF-Schema, and the stored schema typing is correct, then $n = m = 1$ and the condition $T_i \preceq S_j$ is *true*. Consequently, Rule 22 generates exactly one minimal type.

KGs with the contextual representation. The collective findings of the research in the areas of Cognitive science [32, 13] show that natural language is inherently contextual, and the context is essential in the human representation of knowledge and reasoning. Similarly, in the area of AI reasoning [19, 9, 5], the context is essential because the interpretation of propositions, predicates, and defaults often depend on the local assumptions and domain in which they are applied.

Most recent KGs enforce exactly one meaning of a predicate in KGs through the RDF Schema. On the other hand, the contextual representation and reasoning allow the definition of multiple senses of a predicate. There are many motivations for adopting contextual representation and reasoning in a KG. First, with the evolution of KGs, there are many examples where a KG is represented in a modular way, splitting the dataset into parts that correspond to the contexts. The meaning of a predicate can be different in different contexts, while the reasoner is able to disambiguate among the different meanings of a predicate. The examples of KGs using contexts include the named graphs in DBpedia [1], Wikidata [34], and Yago [12]. Another example is Cyc [25] that uses microtheories to represent different contexts. Similar to Cyc, Scone [7] is a KR system that can define spaces (contexts) and uses contextual reasoning. Finally, the $\wedge$ and $\vee$ -types are often implemented in KGs in the form of OWL type constructors `owl:intersectionOf` and `owl:unionOf`. The OWL union and intersection type constructors are employed mostly in domain-specific KGs like biomedical and genomic ontologies, where new classes can be defined as logical combinations of existing classes.

To be able to study the behavior of KG predicates with multiple senses in the presence of triple types, we propose a minimal KG schema language that includes the triple types stored in a KG as triples. If there is more than one schema type with a predicate p , then they are treated as alternatives.

Example 3. Suppose a KG includes the schema types (T_1, p, T_2) and (T_3, p, T_4) , where $T_j^{j=1..4} \in \mathcal{T}_i$. The schema type of ground triples with p is $(T_1, p, T_2) \vee (T_3, p, T_4)$. $\square$

Further, the proposed KG schema language can use $\wedge$ - $\vee$ types in subject and object components of triple types. The use of $\vee$ -type in place of the domain or range type is redundant since the disjunction can be expressed with multiple triple types. Hence, the *minimal KG schema language* includes solely the triple types of the form $(\wedge_{i=1}^n S_i, p, \wedge_{j=1}^m T_j)$, where $S_i^{i=1..n} \in \mathcal{T}_c$ and $T_j^{j=1..m} \in \mathcal{T}_l \cup \mathcal{T}_c$.

Let us now present the rules that derive the schema type of a ground triple t in the case our minimal KG schema language is used. First, the following Rule 23 generates all valid alternatives of stored triple types given a triple t .

$$\frac{\begin{array}{c} t \in \mathcal{V}_t, t = (s, p, o) \\ T_i^{i=1..n} \in \mathcal{T}_t, T_i = (D_i, p, R_i) \quad T_i^{i=1..n} \in \Delta \end{array}}{t :_{\uparrow} \vee_{i=1}^n T_i} \quad (23)$$

The above rule states that the *stored types* of t are the triple types $T_i \in \mathcal{T}_t \wedge T_i \in \Delta$ including a predicate p . Note that D_i and R_i are the types of domains and ranges of p that can stand for an $\wedge$ -type. A triple type $\vee_{i=1}^n T_i$ of t is a disjunction of all valid stored triple types of t .

Similar to Rule ??, which is defined to find the minimal types of a set of classes, the following Rule 24 generates the disjunction of minimal schema types.

$$\frac{t \in \mathcal{V}_t \quad t :_{\uparrow} \bigvee_{i=1}^n T_i}{t :_{\uparrow} \bigvee \{S \mid S \in \{T_i\}_{i=1}^n \wedge \forall i = 1..n, S \preceq T_i \vee S \not\sim T_i\}} \quad (24)$$

The rule says that the triple types S are minimal schema types of a schema type $\bigvee_{i=1}^n T_i$. S is minimal since all other types T_i are either more general or equal ($\succeq$), or not related to S .

5.4 Typing a triple

Before we present the final step in typing a triple t , an overview of the work done so far is given. First, we derive the ground type of the triple t and then infer the minimal upper bound of the ground type. The ground typing inspects all ground types of t 's components. In case the MUB type is close to the $\top$ type, then there is an outlier in the set of ground types that can be revealed in the computation of the MUB type.

Second, the schema type of t is derived from the schema of a KG. In case we use RDF-Schema semantics of a KG, then the rules infer a single minimal schema type. If there is more than one schema type, then the domains and ranges from the different definitions are merged into one $\wedge$ -type. At this point, we can not verify if there are errors in types from the domain and/or range of a predicate. In case we use RDF-Schema with $\wedge$ - $\vee$ types, then the rules can infer multiple disjunctive schema types. Also in this case, there are no additional constraints that could be verified.

Let us now present the final typing of t by relating the ground type of t with the minimal schema type of t via the subtyping relation $\preceq$. If the RDF-Schema semantics are used, then we have a single schema type which should be related to the ground type of t . The following Rule 25 derives the final type of t under RDF-Schema semantics.

$$\frac{t \in \mathcal{T}_i \quad T \in \mathcal{T}_t, t :_{\downarrow} T \quad S \in \mathcal{T}_t, t :_{\uparrow} S \quad T \preceq S}{t : S} \quad (25)$$

The type of t is computed by first deriving the ground type T and the schema type S of t . S is the final type of t if T is a subtype of S . In case $T \not\preceq S$, then the ground type $T = \bigwedge [T_i^{i \in [1, n]}]$ includes at least one $T_i \not\preceq S$.

Let's now assume that we use the *minimal KG schema language*, as defined in Section 5.3, for the representation of the KG schema. Note also that we have to use Rules 23-24 in order to derive the schema triple type. The rule for selecting the final type of t is now as follows.

$$\frac{t \in \mathcal{T}_i \quad t :_{\downarrow} T \quad t :_{\uparrow} \bigvee_{j=1}^m S_j}{t : \bigvee_{j=1}^m \{S_j \mid T \preceq S_j\}} \quad (26)$$

Only those types S_j that are linked to T via the poset hierarchy of types are gathered in the final type. This means that the ground triple selects, via its type, the meaning of the predicate p .

The examples of final types that include more than one alternative schema type with different meanings can appear when typing triple patterns. In case a triple pattern

tp contains one or more variables, then the ground type of one or more tp 's components can not be derived; hence, the $\top$ type is used. Consequently, the ground type of the triple pattern may match more than one schema type.

6 Related work

Most of the related work is cited in the text when presenting the particular topic. In this section, we present only the work that is not directly related to the presented work but represents a contribution to the typing of the knowledge graphs.

The entity typing and type inference deal with predicting and inferring the types (classes) of entities that are either missing or incorrect. The automatic type assignment can use logic-based assignment where a reasoner infers the type of an entity from the schema [15]. Alternatively, the rule-based inference can be employed on the rules defined as a schema by knowledge engineers [14]. Reasoners use them automatically. Another alternative is the use of ML-based type prediction [35]. Various techniques can be used, like entity embeddings and graph neural networks, to produce embeddings to be fed into the classifier.

The schema-based type checking is about the verification of RDF-Schema [33] and OWL [10] rules and constraints against the data (ABox) and schema (TBox) parts of a KG [2, 15, 20]. The domain and range types of predicates are checked to determine whether they are correctly interpreted among the ground triples of a KG. The consistency of subtyping and inheritance is verified in the KG schema and in the data. Similarly, the disjointness of types and other OWL constraints is checked. Most of the presented themes are covered by tools based on SHACL [16] and ShEx [3].

Type checking in query answering ensures that the variables and results of a query over a KG are consistent with the schema of a KG [37, 36]. A type-checker for queries requires that a query respects class hierarchies, domain/range constraints, and disjointness rules. All type manipulation in [37, 36] is at the level of SparQL [8] variables. This means, for example, that triple patterns do not have type, so there is no way to reason about typing of triples. Also, the union and intersection types are used directly, without the means to manipulate $\wedge$ - $\vee$ type expressions.

The type information can also be used to improve the query optimization and execution. In [18], they propose to leverage type information to optimize query execution and filter semantically invalid results. In most cases, the presented approaches use fragments of types that are adapted for the particular problem.

7 Conclusions

The implementation of typing a knowledge graph is currently underway in the context of a working prototype RDF store *epsilon* [29], which is used for browsing and querying knowledge graphs, particularly the schema component. *epsilon* has also been used as a prototype environment for computing type-based statistics of KGs based on types [31].

The *epsilon* environment includes a simple language for the manipulation of the sets of identifiers and triples. Algebraic operations are used for mapping sets to new sets by

using single predicates, path expressions, or, transitive closure operations with fine control over the computation of closures. Additionally, operations on posets of identifiers and triples are being implemented to compute minimal and maximal elements, the MUB and MLB as well as LUB and GLB. The typing rules are implemented as procedures within this framework.

8 Acknowledgments

The author acknowledges the financial support from the Slovenian Research Agency (research core funding No. P1-00383).

References

1. S. Auer, C. Bizer, G. Kobilarov, J. Lehmann, R. Cyganiak, and Z. Ives. Dbpedia: A nucleus for a web of open data. In *The Semantic Web*, volume 4825 of *Lecture Notes in Computer Science*, pages 722–735. Springer, 2007.
2. F. Baader, D. Calvanese, D. McGuinness, D. Nardi, and P. Patel-Schneider. *Description Logic Handbook*. Cambridge University Press, 2002.
3. I. Boneva, J. E. L. Gayo, E. G. Prud’hommeaux, and S. Staworko. Shape expressions schemas, 2015.
4. R. J. Brachman and J. G. Schmolze. An overview of the kl-one knowledge representation system. *Cognitive Science*, 9:171–216, 1985.
5. G. Brewka, F. Roelofsen, and L. Serafini. Contextual default reasoning. In *Proceedings of the 20th International Joint Conference on Artificial Intelligence (IJCAI 2007)*, pages 268–273, Hyderabad, India, 2007. Morgan Kaufmann Publishers.
6. B. A. Davey and H. A. Priestley. *Introduction to lattices and order*. Cambridge University Press, 2nd edition, 2002.
7. S. E. Fahlman. Using scone’s multiple-context mechanism to emulate human-like reasoning. In *Advances in Cognitive Systems, Papers from the 2011 AAAI Fall Symposium, Arlington, Virginia, USA, November 4-6, 2011*, volume FS-11-01 of *AAAI Technical Report*. AAAI, 2011.
8. S. H. Garlik, A. Seaborne, and E. Prud’hommeaux. SPARQL 1.1 Query Language. <http://www.w3.org/TR/sparql11-query/>.
9. C. Ghidini and F. Giunchiglia. Local models semantics, or contextual reasoning = locality + compatibility. *Artificial Intelligence*, 127(2):221–259, 2001.
10. O. W. Group. Owl 2 web ontology language. W3C Recommendation, Dec. 2012.
11. J. R. Hindley. *Basic simple type theory*. Cambridge University Press, USA, 1997.
12. J. Hoffart, F. M. Suchanek, K. Berberich, and G. Weikum. Yago2: A spatially and temporally enhanced knowledge base from wikipedia. *Artificial Intelligence*, 194(0):28 – 61, 2013. Artificial Intelligence, Wikipedia and Semi-Structured Resources.
13. D. L. Hollister, A. J. Gonzalez, and J. Hollister. Contextual reasoning in human cognition and its implications for artificial intelligence systems. In *Modeling and Using Context (CONTEXT 2017)*, volume 10257 of *Lecture Notes in Computer Science*, pages 599–608. Springer, 2017.
14. I. Horrocks, P. F. Patel-Schneider, H. Boley, S. Tabet, B. Grosz, and M. Dean. Swrl: A semantic web rule language combining OWL and ruleml. In *W3C Member Submission*, 2004.

15. I. Horrocks, P. F. Patel-Schneider, and F. van Harmelen. From SHIQ and RDF to OWL: The making of a web ontology language. *Journal of Web Semantics*, 1(1):7–26, 2003.
16. H. Knublauch and D. Kontokostas. Shapes constraint language (SHACL). World Wide Web Consortium (W3C) Recommendation, July 2017. Accessed: 2024-09-07.
17. S. B. Knudstorp. Tho modal logic of minimal upper bounds, 2024.
18. I. Kolli and B. Glimm. Optimizing sparql query answering over owl ontologies. *Journal of Artificial Intelligence Research*, 48:253–303, 2013.
19. J. McCarthy. Notes on formalizing context. In *Proceedings of the 13th International Joint Conference on Artificial Intelligence (IJCAI 1993)*, pages 555–560, Chambéry, France, 1993. Morgan Kaufmann.
20. B. Motik, P. F. Patel-Schneider, and B. Parsia. OWL 2 web ontology language: Structural specification and functional-style syntax. W3C Recommendation, Dec. 2012.
21. Owl 2 web ontology language. <http://www.w3.org/TR/owl2-overview/>, 2012.
22. B. C. Pierce. Programming with intersection types, union types, and polymorphism, 1991.
23. B. C. Pierce. Intersection types and bounded polymorphism, 1996.
24. B. C. Pierce. *Types and Programming Languages*. MIT Press, 1 edition, Feb. 2002.
25. D. Ramachandran, P. Reagan, and K. Goolsbey. First-orderized researchcyc: Expressivity and efficiency in a common-sense ontology. In *AAAI Reports*. AAAI, 2005.
26. Resource description framework (rdf). <http://www.w3.org/RDF/>, 2004.
27. Rdf schema. <http://www.w3.org/TR/rdf-schema/>, 2004.
28. Rdf schema. <https://www.w3.org/TR/rdf-mt/>, 2004.
29. I. Savnik. Epsilon: Lightweight rdf store (working prototype). <https://github.com/iztok-savnik/epsilon>, 2025. GitHub repository.
30. I. Savnik. Typing graph patterns. Technical Report (In preparation), FAMNIT, University of Primorska, 2025.
31. I. Savnik, K. Nitta, R. Skrekovski, and N. Augsten. Type-based computation of knowledge graph statistics. *Annals of Mathematics and Artificial Intelligence*, 2025.
32. K. Stenning and M. van Lambalgen. *Human Reasoning and Cognitive Science*. MIT Press, Cambridge, MA, 2008.
33. D. Tomaszuk and T. Haudebourg. Rdf 1.2 schema. W3C Working Draft, Apr. 2024.
34. D. Vrandečić and M. Krötzsch. Wikidata: A free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85, 2014.
35. Y. Yaghoobzadeh, H. Adel, and H. Schütze. Corpus-level fine-grained entity typing. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, pages 940–949, 2018.
36. L. Zhang, Y. Zhao, and L. Zhang. Entity type saturation: A technique for type checking in sparql query answering. *Semantic Web*, 10(1):1–19, 2019.
37. Y. Zhao, H. Wang, L. Zhang, and L. Zhang. Type checking and testing of sparql queries. In *Proceedings of the 16th International Semantic Web Conference (ISWC 2017)*, pages 526–541. Springer, 2017.